Repeat-Viewing-Aware Top-5 TV Program Recommendation on SWEET.TV: CBF, Matrix Factorization and a Schedule Prior

Student: Ilya Bochkarev (Бочкарев Илья) Team: Individual Email: ilya.bochkarev87@outlook.com Date: 2026-10-25
Assignment: LLM4Rec Midterm Quiz — TV Program Recommendation | Kaggle: IlyaBochkarev | GitHub: github.com/bochkarev87/sweettv-tv-program-recommender

Abstract. We rank five TV programs for each of 1,260 SWEET.TV users for weeks 31–42 of 2020 from viewing logs and channel schedules. Logs are joined to the schedule by channel-time overlap, and popularity, content-based filtering (CBF), implicit ALS and PureSVD are compared under one temporal split (train before 1 June, validate 1 June–26 July) with MAP@5. The final hybrid combines a decayed own-history score, SVD and a schedule-based airing prior; it reaches validation MAP@5 0.2966 (popularity 0.167) and a Kaggle private score of 0.30404. Because 67% of target programs were already watched by the same user, personal history and schedule exposure matter far more than content metadata (CBF 0.0022).

Index Terms — TV recommendation, implicit feedback, matrix factorization, content-based filtering, temporal validation

1. Introduction

Problem. Given SWEET.TV viewing sessions (user, channel, start, stop; 9 Mar–26 Jul 2020) and channel schedules with program metadata, output an ordered Top-5 of `tv_show_id` for every test user; quality is MAP@5 against the programs the user preferred in weeks 31–42 [5]. **Motivation.** Live TV has a fixed catalogue of recurring programs, so a recommender must decide between "your usual shows" and discovery, and must know what will actually be on air. **Example.** For a typical validation user (234 programs watched) the final model returns news and talk shows the user watched most (e.g. 240081 "Факты. День.", 6500479 "Мир наизнанку") and hits 3 of 5 target programs (AP@5 0.420; the v1 hybrid hits 4, AP@5 0.643; Table 3). **Contributions.**

- A documented log–schedule matching rule and a leakage-free temporal protocol shared by popularity, CBF, ALS, SVD and a tuned hybrid, reporting negative results (CBF, ALS) as they are.
- A schedule-exposure prior (how often a program is on air) that improved the hybrid on validation and on Kaggle, with a sensitivity check on the validation horizon.

2. Related Work

Prior work consulted (what I took from it, in my own words).

- **[1] Hu, Koren, Volinsky (2008).** In implicit data a large value, for example a long viewing time, does not mean that the user likes the item more: it only raises our confidence that the user likes it. The authors therefore split every observation into a binary preference and a confidence weight and train the model on all user–item pairs, not only the observed ones. I used the same idea in my ALS model: α, λ, the number of factors and the number of iterations play the same role as in the paper.
- **[2] Cremonesi, Koren, Turrin (2010).** The main thing I took from this work is that top-N recommendation quality should be measured by ranking accuracy, not by rating-prediction error, so I evaluate my models with ranking metrics. The authors also show that a simple non-personalised method can be as good as complex ones, because a few very popular items strongly influence the lists. This agrees with my result: the popularity baseline scored 0.167, and I treat this number as the bar that any personalised model has to beat. In the section on PureSVD the authors propose a very simple model: missing ratings are replaced by zeros, truncated SVD is applied to the resulting matrix, and recommendations are built from the reconstructed row of the user. They show that such a model, which does not try to fit only the known values, works well precisely on the top-N task. For my project this matters because my SVD is built in a similar way, and I compare it with ALS, which takes the confidence in the observations into account explicitly.
- **[3] Koren, Bell, Volinsky (2009).** This article helped me to see matrix factorization as one picture: users and items lie in the same low-dimensional space and the predicted score is the dot product of their vectors; the scheme can be extended to implicit feedback with confidence levels. So I can regard my ALS and SVD models as two variants of one approach and compare them fairly in the project.
- **[4] Lops, de Gemmis, Semeraro (2010)**, chapter "Content-based Recommender Systems: State of the Art and Trends". The authors describe an approach in which each item is represented by a set of features, a user profile is built from the user's past actions, and items similar to the profile are recommended. The main advantages are that it does not need data of other users and can recommend new items; the main drawbacks are a narrow range of recommendations and dependence on the quality of the features.

MAP@K follows the course definition with the $\min(K,|R_u|)$ normalisation [6]. **Alternatives considered:** association rules — not implemented, because with few co-watched pairs per user they were expected to add little over popularity and time was limited; personalized PageRank — not implemented for the same reason (it remains a natural extension); BPR/two-tower models — out of scope for the available time.

3. Method

Approach. We (1) build user–program interactions by joining logs to the schedule, (2) split by time, (3) score every program that is on air in the target window with each method, (4) blend z-scored method scores with weights chosen on a held-out half of the users, and (5) retrain on all logs and predict weeks 31–42.

Matching rule and missing IDs. A session [start, stop] on channel c is matched to every airing on the same channel whose interval [start, start+duration) overlaps it; a (session, program) pair is kept if the overlap is ≥60 s (threshold chosen a priori, not tuned). Per (user, program) we sum overlap seconds and use w=log(1+minutes). Schedule rows with `tv_show_id=0` (irrelevant programs, 88–89% of rows) never become interactions; schedule files overlap at week boundaries and are de-duplicated on (channel, start). Result: 5,821,987 sessions → 3,742,237 interactions, 4,895 users, 4,362 programs; **53.0%** of watch time is matched to relevant programs; the rest falls on programs with `tv_show_id=0`, on the 114 of 202 logged channels that have no relevant program in the schedule (5 channels are absent from it altogether), or on overlaps shorter than 60 s. Log and schedule clocks were assumed equal: shifts of −3…+3 h were tried and zero shift gave the largest share of session starts within 90 s after a program start in both March and July (3.1%/3.0% vs 2.5–2.8%) — a weak but consistent signal.

Protocol. Train: sessions before 1 June 2020; validation: 1 June–26 July (8 weeks). Candidate pool: programs with an airing in the validation window (5,732; the test pool is the 31–42 schedule, 6,646 programs). Ground truth per user: the 5 programs with the largest validation watch time (the competition asks for the 5 "most preferred"). Evaluation users: the 1,244 of the 1,260 submission users with validation activity (7 have no training history and get popularity). Hyper-parameters and blend weights are chosen on a random half A of users and reported on the held-out half B and on all users (seed 42). No validation or future sessions are used for training; program metadata and future schedules are provided inputs.

Key design decisions. (1) *Overlap-based matching* with a 60 s minimum, because sessions often span two programs (Check B) and partial viewing should count in proportion to its length. (2) *Already-watched programs are not excluded*: 67.4% of validation targets are repeats, and excluding them drops every method below 0.01. (3) *Z-scored blend with weights chosen on user-half A*, so that scores of different scales can be combined and the reported number comes from users not used for tuning.

Methods. *Popularity*: number of distinct training users. *Own-history prior* (reference, not one of the two required methods): decayed watch time (half-life 56 d), log-scaled. **CBF**: TF-IDF blocks for category, genres, actors, director, title words and decade (weights 1/1/0.5/0.5/0.5/0.3); user profile = watch-weighted sum of program vectors, cosine ranking. **MF**: implicit ALS [1] (confidence $1+\alpha w$) and PureSVD [2] of the user–program matrix of w. *Hybrid*: $\Sigma\, w_k \cdot z(S_k)$ with per-user z-scores over the pool; *v2* adds the airing prior $z(\log(1+\#\text{airings of the program in the target window}))$, known in advance from the schedule. **Tools:** Python 3.14.5, pandas 3.0.6, NumPy 2.5.3, SciPy 1.18.1 (`svds`), scikit-learn 1.9.1 (`TfidfVectorizer`); ALS is our own implementation (no `implicit` package). CPU only, one Windows 11 PC. Seeds: ALS init 0, user split 42.

4. Experiments

Setup. SWEET.TV competition data [5]; protocol above; MAP@5 with normalisation $\min(5,|R_u|)$ [6]. Of the validation targets, 67.4% were already watched by the user in training and 12.3% were never seen in training by anyone (cold programs).

Method	Configuration	MAP@5 (B, held-out)	MAP@5 (all)	HR@5	P@5	fit, s	rank 1,244 users, s
Popularity (#users)	all train	0.1746	0.1673	0.683	0.226	0.0	0.08
Popularity, last 28 d	28-day window	0.1773	0.1694	0.683	0.226	0.0	0.08
Own-history prior (ref.)	half-life=56 d	0.2897	0.2838	0.814	0.349	0.0	0.72
Airing prior alone (ref.)	log(1+#airings in window)	0.0299	0.0270	0.129	0.031	0	0.14
CBF (required method 1)	TF-IDF: cat/genre/actor/dir/title/decade	0.0026	0.0022	0.019	0.004	0.0	0.65
CBF + show-identity feature	identity weight 2	0.0045	0.0042	0.031	0.008	0	0.63
MF: ALS (required method 2a)	k=64 alpha=0.5 reg=30 it=8	0.1228	0.1174	0.613	0.182	5.0	0.16
MF: PureSVD (required method 2b)	rank=128	0.2545	0.2476	0.811	0.318	0.6	0.15
Hybrid v1 (history+SVD)	w=2 / 0.5 (z-scored)	0.2949	0.2879	0.828	0.352	0.0	0.14
Hybrid v2 = v1 + airing prior (final)	prior w=1, log #airings	0.3046	0.2966	0.838	0.361	0	0.14

Table 1. Validation results (train < 1 Jun, validate 1 Jun–26 Jul 2020). Configurations of ALS/SVD/history were selected on user-half A; times are single measured runs.

Comparison with baseline. The two required methods behave very differently: PureSVD (0.248) is far above popularity (0.167), ALS (0.117) is below it, and CBF (0.0022) is almost zero. Own history alone (0.284) already beats every single required method; the hybrid v1 reaches 0.2879 and v2 0.2966 (held-out half B: 0.2949 → 0.3046; without the prior at the same weights 0.2949). Recommending only unseen programs removes the signal: popularity 0.0088, history 0.0088, SVD 0.0039 (MAP@5, all users).

Check	without airing prior	with airing prior
8-week validation (1 Jun–26 Jul), B / all	0.2949 / 0.2879	0.3046 / 0.2966
12-week validation (4 May–26 Jul, same horizon as test), B / all	0.3218 / 0.3288	0.3369 / 0.3439
Kaggle (weeks 31–42): private / public	0.25025 / 0.28799 (v1)	**0.30404 / 0.33326 (v2, final)**

Table 2. Effect of the airing prior on validation and on Kaggle.

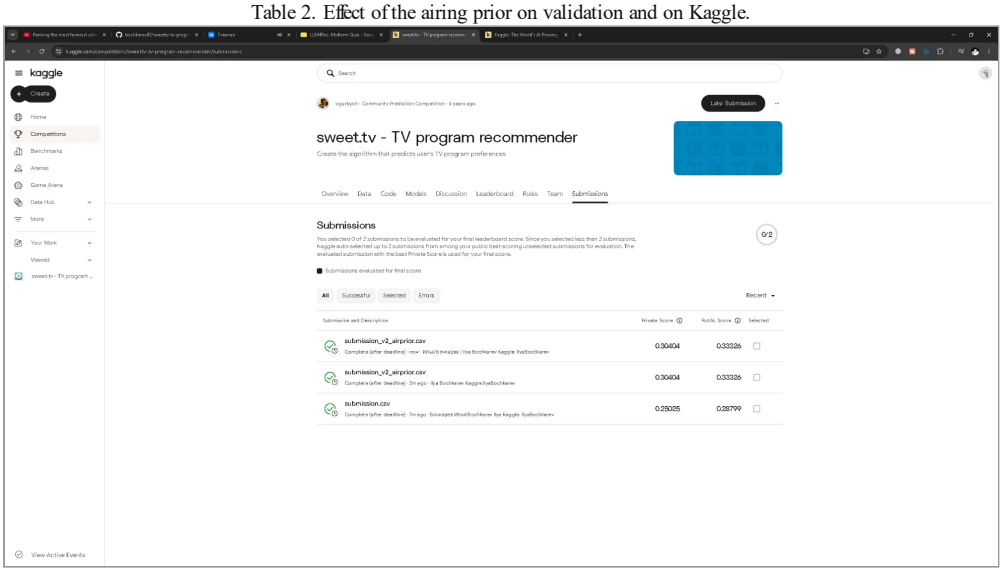


Figure 1. Kaggle Submissions page (screenshot): v1 `submission.csv` scored 0.25025 private / 0.28799 public; the final designated submission `submission_v2_airprior.csv` (uploaded twice; the top entry carries the Russian name in its description) scored **0.30404 private / 0.33326 public**. The final file is in the repository.

Verification. (a) the `apk` code reproduces the worked example of the course slides [6] (0.917 and 0.533; `unit_checks_output.txt`); (b) `09_unit_checks.py` asserts that training sessions end before the cutoff, validation sessions start after it, and targets lie inside the candidate pool; (c) CBF sanity check `08_cbf_sanity.py`: raising the show-identity weight to 100 lifts CBF to 0.279, almost the own-history level (0.284), so the CBF pipeline is correct and its weakness is a property of the metadata; (d) both submission files are checked against the template (1,260 users, 5 distinct programs each, all inside the weeks 31–42 schedule); outputs of all checks are committed (`unit_checks_output.txt`, `cbf_sanity_output.txt`). (e) **Manual checks by the student** (Figure 2): *Check A* — for user A (Hybrid v1, validation) the relevant programs are at ranks 1, 3, 4, 5 of 5 relevant, AP@5 = (1/1+2/3+3/4+4/5)/5 = 0.6433, equal to the code output; *Check B* — for one raw session (channel 3, 2020-04-10 16:48:21–17:16:54) the overlaps with the scheduled programs were computed by hand: 0 s (15:25–16:45), 1299 s (“ТСН”, 17:10:00−16:48:21) and 414 s (“Великолепный век”, 17:16:54−17:10:00); they equal the values stored by the pipeline, and 1299+414 = 1713 s equals the session length (28 min 33 s); *Check C* — the worked example of slide 5 [6] computed by hand: method X (relevant programs at ranks 1, 2, 4) gives (1+1+0.75)/3 = 0.917 and method Y (ranks 2, 4, 5) gives (0.5+0.5+0.6)/3 = 0.533, equal to the slide and to the code (Figure 3); (f) *Re-run from scratch* — running `04_final_submission.py air` again regenerated `submission_v2_airprior.csv` with the identical SHA-256 (5EDF8225A174248D…91D15ECF) before and after the run; (g) *Formula check*

— the student compared `_solve_side` with the user-vector solution of [1], $x_u = (Y^T C^u Y + \lambda I)^{-1} Y^T C^u p(u)$: the code builds $A = Y^T Y + \lambda I + Y^T(C^u - I)Y$ (since $Y^T C^u Y = Y^T Y + Y^T(C^u - I)Y$) and the right-hand side $Y_i^T(1 + c_{ui} - 1)$ over observed items only, which equals $Y^T C^u p(u)$ because $p_{ui} = 1$ for observed pairs and 0 otherwise.

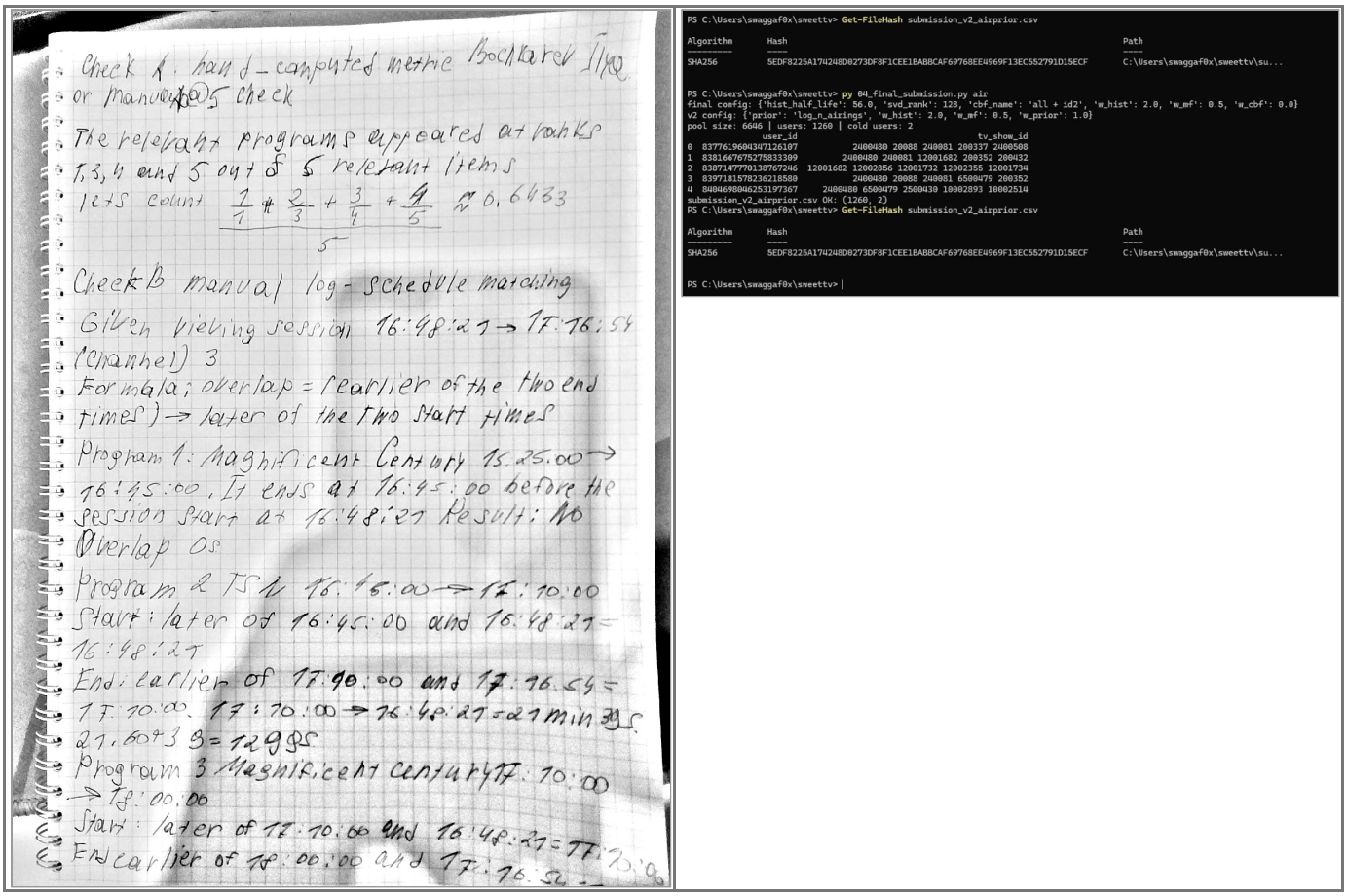


Figure 2. Left: the student's hand computations, Check A (AP@5) and Check B (log–schedule overlap). Right: console screenshot of check (f), `Get-FileHash` before and after `py 04_final_submission.py air`; the SHA-256 of `submission_v2_airprior.csv` is identical.

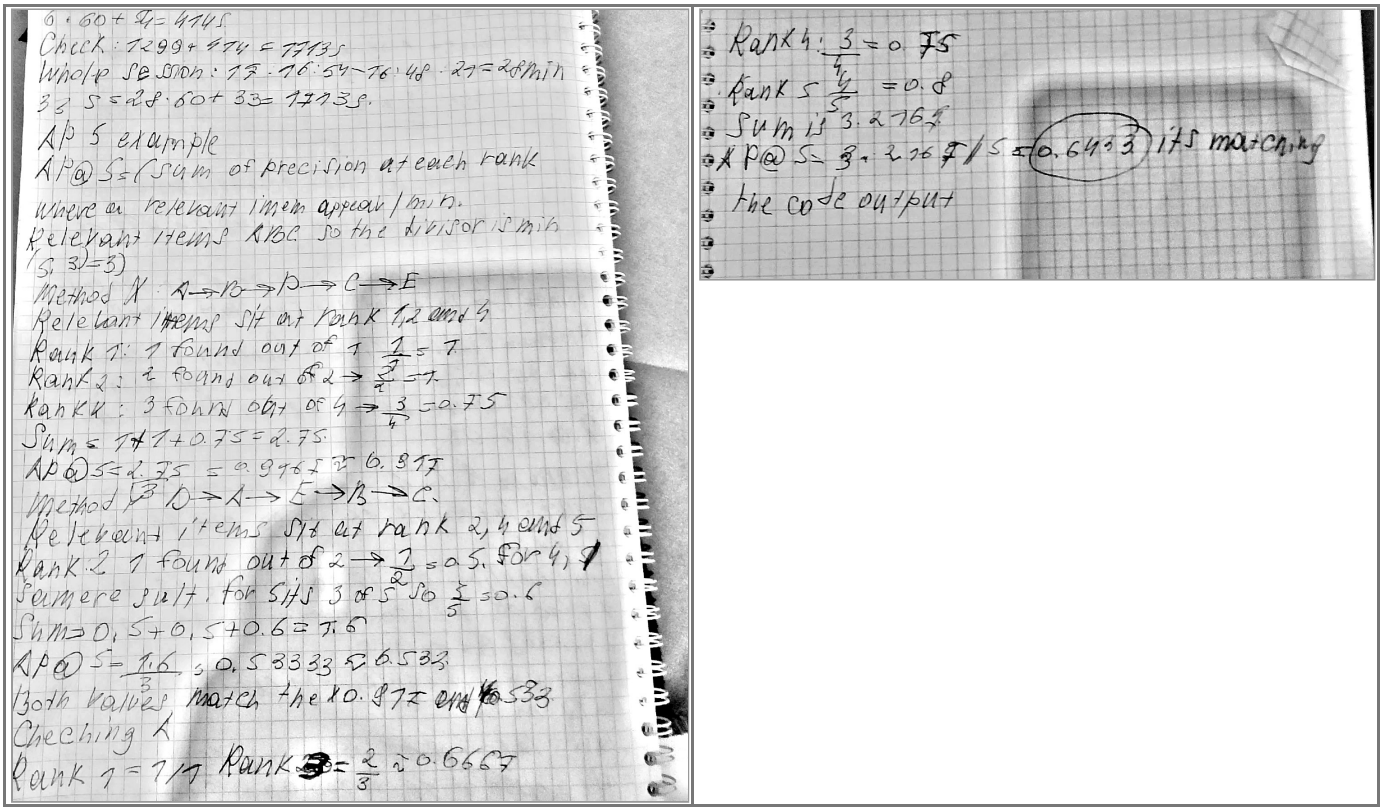


Figure 3. Student's hand computation of the slide-5 example (methods X and Y, 0.917 and 0.533) and the continuation of Check A (0.6433, matching the code output).

5. Discussion

5.1 Business and algorithmic analysis

Selected strategy for SWEET.TV: hybrid v2 — decayed own-history score + SVD collaborative signal + airing prior, with CBF kept out of the final blend (weight 0 was selected on half A). **Why (validation):** 0.2966 MAP@5 vs 0.167 for popularity and 0.248 for the best required

method; the gain of the airing prior is 0.2949 → 0.3046 on B at 8 weeks and 0.3218 → 0.3369 at the 12-week horizon of the test. **Why (cost):** SVD fits in 0.6 s and ALS in 5.0 s on the 4.9k×9.7k matrix, and ranking 1,244 users takes well under a second per method (Table 1), so the model can be retrained after every schedule update; CBF needs item-attribute preprocessing but is cheap too. **Why (examples):** Table 3.

Users. We took the user closest to the median activity (234 distinct programs in training) as the typical case and the user closest to the 10th percentile (97 programs) as the light case; the rule is applied before looking at any recommendations, and the same two users are shown for every method.

User A (typical: median activity): user …533863, 234 distinct shows / 293 h in training. **Train top-5 by watch time:** 240081 Факты. День *(Инфо)*; 2400480 ТСН. Неделя *(Инфо)*; 6500479 Мир наизнанку. Брази… *(Познав)*; 2500413 Гражданская оборона *(Инфо)*; 20088 Завтрак с 1+1'. Инфо… *(Развле)*. **Validation truth:** 240081 Факты. День *(Инфо)*; 6500479 Мир наизнанку. Брази… *(Познав)*; 200469 Варьяты. Лучшее *(Развле)*; 100076 Т/с 'Комиссар Рекс',… *(Сериал)* **[new]**; 2500413 Гражданская оборона *(Инфо)*.

#	Popularity AP@5=0.200	CBF AP@5=0.000	ALS AP@5=0.333	SVD AP@5=0.333	Hybrid v1 AP@5=0.643	Hybrid v2 (final) AP@5=0.420
1	2400480 ТСН. Неделя *(Инфо)*	90080647 Х/ф 'Джуманджи: Зо… *(Фильмы)*	2500413 Гражданская оборон… *(Инфо)* ✓	240081 Факты. День *(Инфо)* ✓	240081 Факты. День *(Инфо)* ✓	240081 Факты. День *(Инфо)* ✓
2	240081 Факты. День *(Инфо)* ✓	90081466 Х/ф 'Такси 5 *(Фильмы)*	200432 Дизель-шоу. 8 март… *(Развле)*	2400480 ТСН. Неделя *(Инфо)*	2400480 ТСН. Неделя *(Инфо)*	2400480 ТСН. Неделя *(Инфо)*
3	2400508 Сегодня. Итоги с О… *(Инфо)*	90081679 Премьера. Х/ф 'Мог… *(Фильмы)*	200469 Варьяты. Лучшее *(Развле)* ✓	200469 Варьяты. Лучшее *(Развле)* ✓	6500479 Мир наизнанку. Бра… *(Познав)* ✓	20088 Завтрак с 1+1'. Ин… *(Развле)*
4	6500479 Мир наизнанку. Бра… *(Познав)* ✓	90082810 Премьера. Х/ф 'Шаз… *(Фильмы)*	10003014 Т/с 'Отдел 44', 71… *(Сериал)*	700389 Кто сверху?, 6 сез… *(Развле)*	200469 Варьяты. Лучшее *(Развле)* ✓	6500479 Мир наизнанку. Бра… *(Познав)* ✓
5	2400467 Викна-Новости *(Инфо)*	90076295 Х/ф 'Призрачный па… *(Фильмы)*	300358 Скетч-шоу 'На трои… *(Развле)*	20088 Завтрак с 1+1'. Ин… *(Развле)*	2500413 Гражданская оборон… *(Инфо)* ✓	200469 Варьяты. Лучшее *(Развле)* ✓

User B (light: 10th-percentile activity): user …572275, 97 distinct shows / 111 h in training. **Train top-5 by watch time:** 400361 МастерШеф', 7 сезон,… *(Познав)*; 400357 Холостяк', 10 сезон,… *(Развле)*; 12001682 М/с 'Губка Боб Квадр… *(Детям)*; 2400480 ТСН. Неделя *(Инфо)*; 700389 Кто сверху?, 6 сезон… *(Развле)*. **Validation truth:** 2400480 ТСН. Неделя *(Инфо)*; 20088 Завтрак с 1+1'. Инфо… *(Развле)*; 12001682 М/с 'Губка Боб Квадр… *(Детям)*; 90081234 Х/ф 'Кавалерия *(Фильмы)* **[new]**; 90044282 Х/ф 'Разборка в Брон… *(Фильмы)* **[new]**.

#	Popularity AP@5=0.200	CBF AP@5=0.000	ALS AP@5=0.333	SVD AP@5=0.400	Hybrid v1 AP@5=0.067	Hybrid v2 (final) AP@5=0.453
1	2400480 ТСН. Неделя *(Инфо)* ✓	90079463 Х/ф 'Братья из Гри… *(Фильмы)*	20088 Завтрак с 1+1'. Ин… *(Развле)* ✓	2400480 ТСН. Неделя *(Инфо)* ✓	400361 МастерШеф', 7 сезо… *(Познав)*	2400480 ТСН. Неделя *(Инфо)* ✓
2	240081 Факты. День *(Инфо)*	90079323 Х/ф 'Иллюзия обман… *(Фильмы)*	700389 Кто сверху?, 6 сез… *(Развле)*	20088 Завтрак с 1+1'. Ин… *(Развле)* ✓	400357 Холостяк', 10 сезо… *(Развле)*	400361 МастерШеф', 7 сезо… *(Познав)*
3	2400508 Сегодня. Итоги с О… *(Инфо)*	90076295 Х/ф 'Призрачный па… *(Фильмы)*	2400480 ТСН. Неделя *(Инфо)* ✓	400361 МастерШеф', 7 сезо… *(Познав)*	2400480 ТСН. Неделя *(Инфо)* ✓	12001682 М/с 'Губка Боб Ква… *(Детям)* ✓
4	6500479 Мир наизнанку. Бра… *(Познав)*	90082810 Премьера. Х/ф 'Шаз… *(Фильмы)*	400335 Хата на тата', 7 с… *(Развле)*	400357 Холостяк', 10 сезо… *(Развле)*	700389 Кто сверху?, 6 сез… *(Развле)*	400357 Холостяк', 10 сезо… *(Развле)*
5	2400467 Викна-Новости *(Инфо)*	90072921 Х/ф 'Большой солда… *(Фильмы)*	200469 Варьяты. Лучшее *(Развле)*	200469 Варьяты. Лучшее *(Развле)*	200469 Варьяты. Лучшее *(Развле)*	20088 Завтрак с 1+1'. Ин… *(Развле)* ✓

Table 3. Top-5 of every method for the two users (ID, title, category; ✓ = program in the validation top-5; "[new]" = not in the user's training history). AP@5 per method in the header.

How and why the methods differ. Popularity gives everyone the same national news/talk programs; it is right only when the user happens to watch them. CBF returns programs that are close to the profile in metadata (for both users all five are films, "Фильмы"), which gives novelty but zero hits: a profile averaged over 100–250 programs is dominated by broad categories, and the profile–item cosine of a generic film (0.455) exceeds that of the programs actually watched next (0.14–0.42) for one diagnosed validation user with 146 programs in history (`cbf_sanity_output.txt`). SVD reconstructs the user's own weighted history plus co-watched programs and so returns the habitual programs; ALS, tuned with a small confidence α and strong regularisation, smooths more towards globally popular programs (a likely, untested explanation of its lower score). The final hybrid ranks habitual programs first and uses the airing prior to prefer programs that are broadcast often. **Implication for UX:** the list is a precise "continue your usual shows" shelf with low novelty; a discovery shelf needs a separate objective because excluding already-watched programs drops every method below 0.01.

Limitations and failure cases (supported by results). (1) Personalisation vs exposure is a trade-off per user: for light user B the v1 hybrid scores AP@5 0.067 (SVD 0.400) because it repeats training programs ("МастерШеф", "Холостяк") that are not in the validation top-5, and two of B's five targets are films B had not watched before; the airing prior lifts v2 to 0.453, but for typical user A it lowers AP@5 from 0.643 to 0.420. History cannot predict new programs at all, and they are 12.3% of all targets. (2) CBF fails on generic metadata (Table 1). (3) 47% of watch time is not matched to a relevant program (id 0, channels without relevant IDs, short overlaps). (4) Targets are our own definition of "most preferred" (top-5 by watch time); the private Kaggle score of v1 (0.250) is below its validation value (0.288), while v2 gains much more on Kaggle (+0.054) than on validation (+0.009 at 8 weeks, +0.015 at 12 weeks), which we cannot fully explain. (5) One random user split, no confidence intervals; ALS/SVD hyper-parameter grids are small and the best SVD rank (128, the largest tried) and ALS alpha (0.5, the smallest tried) lie at the grid edge.

5.2 Failure case, root cause and verification (student)

Failure. CBF scored MAP@5 0.002, against 0.167 for popularity. **Root cause.** CBF works correctly, but its metadata are too coarse to single out a specific program. Category and genre are shared by hundreds of programs, so the user profile is blurred: the genre "Фильмы" is summed many times and gets a large weight, while the feature "this exact program" is one of hundreds and individually small. In the diagnosed example (a user with 146 programs in the history) the CBF top-5 scored 0.453–0.457, while the user's target programs (those watched in validation) scored 0.14–0.42. Four of the five recommendations were films from the user's own history: films from the profile outranked the target programs, apparently because the category "Фильмы" dominates the profile. This is one user, and I do not claim it always happens.

Fix and verification. In an experiment with a show-identity weight of 100 (suggested and run by the AI; I checked the results in `cbf_sanity_output.txt`) identity outweighed genre and MAP@5 rose from 0.002 to 0.279, almost the level of the own-history model (0.284). This confirms that the CBF pipeline works. But closeness to 0.284 does not mean that CBF is good: at that weight it essentially became the own-history model.

Why it matters. 67.4% of the targets are programs the user had already watched. The task is determined by repeat viewing of specific programs, and coarse metadata cannot distinguish them. This does not mean metadata are useless: the remaining 32.6% of targets had not been

watched by the user before (for 12.3% of all targets, nobody had watched them in training), and metadata could help there, but I did not test this separately. This is the next improvement.

5.3 What worked (student)

- **Honest temporal split.** Training used data before 1 June, validation used data after. Parameters were selected on one half of the users (A) and the result was read on the other half (B). Results on B and on all users are close (0.2949 and 0.2879), so I see no sign of fitting to the validation data. Manual checks: I computed by hand the overlap of one session with the programs (1299 s and 414 s) and it matched the code; AP@5 for one user (0.6433) matched the code output; and I reproduced the slide-5 example by hand (0.917 and 0.533). The metric code also reproduces the slide example (`09_unit_checks.py`).
- **The method "recommend what the person watched before, with a 56-day decay"** scored 0.2838, while popularity scored only 0.1673. The simple method turned out to be strong, probably because people often return to the same programs (67.4% of the targets are repeats).

5.4 What surprised or did not work (student)

- **67.4% of the targets are repeat views.** If already-watched programs are excluded from the recommendations, everything collapses: popularity 0.0088, history 0.0088, SVD 0.0039. Almost the whole result is explained by the phrase "a person watches what they have already watched".
- **ALS was weaker than popularity.** The best variant gave 0.1174 against 0.1673 for popularity. More factors and a larger alpha made the result worse. I did not check why, so I do not claim a cause.
- **The "how often a program is on air" feature behaved differently.** On Kaggle it gave +0.054 (0.25025 → 0.30404), on my validation only +0.009 (+0.015 at the 12-week horizon). I have not fully explained this gap. For the two analysed users the effect was also different: for user A AP@5 fell from 0.643 to 0.420, for user B it rose from 0.067 to 0.453.

5.5 Next improvement (student)

I would add confidence intervals: repeat the evaluation on several random splits of the users. At the moment I have a single random split and many differences are small (for example hybrid vs history: 0.2949 vs 0.2897 on B). Without intervals I cannot say which differences are real and which are noise. This would show which differences are real and whether the +0.009 validation effect of the airing feature lies within the noise.

6. AI Usage Disclosure

- **AI tools used:** Claude Code (Anthropic desktop app), model Claude Sonnet 5 and, later in the session, Claude Sonnet 5.5; headless Google Chrome only to print the HTML report to PDF.
- **How AI was used:** *code generation* (all scripts in the repository: log–schedule matching, models, experiments, report builder); *debugging* (e.g. a file-encoding false alarm, import paths); *experiment design* (CBF ablation, airing prior, 12-week horizon check); *references* (the AI looked up the four DOIs in Crossref and the Kaggle page to check that the references exist; I then read the relevant parts of [1]–[4], see §2); *docs/viz* (report layout, tables and the text of §1–5.1, which I read and checked against the output files). Sections 5.2–5.5 were written by me; the AI only translated them and corrected wording against the data.
- **What I personally verified:** (1) Check A — AP@5 of one user by hand, 0.6433, equal to the code; (2) Check B — overlap of one raw session with two programs by hand, 1299 s and 414 s, equal to the pipeline, and their sum equals the session length; (3) Check C — the slide-5 example by hand, 0.917 and 0.533; (4) re-ran `04_final_submission.py air` and confirmed an identical SHA-256 of the submission file; (5) read `experiments_output.txt` and `cbf_sanity_output.txt` for the numbers quoted in §5.2–5.4; (6) uploaded the files to Kaggle myself and took the screenshot; (7) read the relevant parts of [1], [2] (including the PureSVD section), [3] and [4] and summarised them in §2; (8) compared the ALS update step `_solve_side` in `models.py` with the formula for the user vector in [1] and found them identical (see §4, check (g)); (9) checked the hyper-parameter grids in `03_experiments.py` against `results_validation.csv`: the best SVD rank (128) is the largest rank tried and the best ALS alpha (0.5) is the smallest tried, so better values outside the grids are possible and I did not extend them.
- **What I trusted without verification:** I did not review the Python code line by line (only the ALS update step `_solve_side` was compared with the formula of [1]) and trusted the AI that the implementations of PureSVD, TF-IDF and the airing prior do what they should (only their outputs were checked, see above); I read only the parts of the papers that are relevant to the project, not the papers in full.
- **Session log reference:** the attached `bochkarev_ilya_midterm_session.json` is the original, unmodified Claude Code log of this session. Its native format is JSON Lines (one JSON object per line); only the file extension is `.json`, as the LMS accepts only `.json` and `.pdf`. It contains every prompt, tool call and file change.

References

[1] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in *Proc. 2008 Eighth IEEE Int. Conf. Data Mining*, 2008, pp. 263–272, doi: 10.1109/ICDM.2008.22. [Accessed: 2026-10-25].

[2] P. Cremonesi, Y. Koren, and R. Turrin, "Performance of recommender algorithms on top-N recommendation tasks," in *Proc. Fourth ACM Conf. Recommender Systems (RecSys '10)*, 2010, pp. 39–46, doi: 10.1145/1864708.1864721. [Accessed: 2026-10-25].

[3] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009, doi: 10.1109/MC.2009.263. [Accessed: 2026-10-25].

[4] P. Lops, M. de Gemmis, and G. Semeraro, "Content-based recommender systems: State of the art and trends," in *Recommender Systems Handbook*, Springer, 2010, pp. 73–105, doi: 10.1007/978-0-387-85820-3_3. [Accessed: 2026-10-25].

[5] "sweet.tv – TV program recommender," Kaggle community prediction competition. [Online]. Available: https://www.kaggle.com/competitions/sweettv-tv-program-recommender. [Accessed: 2026-10-25].

[6] S. Jin, "LLM4Rec – Midterm Quiz," course slides (MAP@K, slides 4–5), HSE University, 2026.